

Taski

A co to?

- **Tasks** w Snowflake służą do harmonogramowania i automatyzacji wykonywania SQL (np. MERGE, INSERT, wywołania procedur składowanych) w określonym czasie lub po zakończeniu innego zadania.
- Działają jako graf zadań (DAG) z jednym zadaniem “root” oraz zależnościami rodzic–dziecko.
- Mogą być oparte na harmonogramie cron lub na tzw. after-trigger (uruchom po zakończeniu rodzica).

Cechy

- Harmonogramowanie: cron lub proste interwały (np. co 15 min).
- Zależności: budowa DAGów dla orkiestracji ETL/ELT.
- Warunki uruchomienia: WHEN (predicate), żeby uruchamiać tylko, gdy spełnione są warunki.

Cechy

- Obsługa zepsutych przebiegów: automatyczne ponowienia i retry policies (na poziomie procedury/SQL zwykle trzeba obsłużyć idempotencję).
- Przypisanie zasobów: WAREHOUSE i opcjonalnie USER TASK MANAGED WAREHOUSE (Snowflake może sam zarządzać magazynem dla tasków).
- Monitoring: INFORMATION_SCHEMA.TASK_HISTORY, TASK_GRAPH(), SHOW TASKS, Query History.

Tworzenie taska

```
CREATE OR REPLACETASK my_hourly_task
```

```
WAREHOUSE = my_wh
```

```
SCHEDULE = 'USING CRON o * * * * Europe/Warsaw'
```

```
AS
```

```
CALL my_schema.my_proc();
```

```
-- Włącz task
```

```
ALTERTASK my_hourly_task RESUME;
```

Tworzenie taska z zależnością (DAG)

```
CREATE OR REPLACE TASK parent_task

WAREHOUSE = my_wh

SCHEDULE = 'USING CRON */15 * * * * Europe/Warsaw'

AS

MERGE INTO dim.customer ...;

CREATE OR REPLACE TASK child_task

WAREHOUSE = my_wh

AFTER parent_task

AS

CALL transform.load_facts();

ALTER TASK parent_task RESUME;

ALTER TASK child_task RESUME;
```

Tworzenie taska z WHEN

```
CREATE OR REPLACE TASK conditional_task
```

```
WAREHOUSE = my_wh
```

```
SCHEDULE = '5 MINUTE'
```

```
WHEN SYSTEM$STREAM_HAS_DATA('my_schema.my_stream') AS
```

```
CALL ingest.apply_changes();
```

SYSTEM\$STREAM_HAS_DATA pozwala uruchamiać tylko, gdy są nowe dane w streamie.

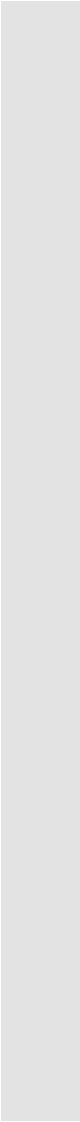
Monitorowanie

- Historia

```
SELECT * FROM TABLE(INFORMATION_SCHEMA.TASK_HISTORY(RERESULT_LIMIT=>50 )) ORDER BY  
SCHEDULED_TIME DESC;
```

- Szybki podgląd

```
SHOWTASKS IN SCHEMA my_database.my_schema;
```

Streams

STREAMS

- Stream w Snowflake to obiekt, który śledzi zmiany w tabeli lub widoku materializowanym.
- Dzięki niemu możesz łatwo pobrać tylko nowe/zmienione/usunięte wiersze od ostatniego “odczytu” strumienia, czyli zrealizować wzorzec Change Data Capture (CDC) bez skomplikowanej logiki.

STREAMS

- **Po co używać streams?**
 - Budowa potoków ETL/ELT, które przetwarzają tylko przyrost danych.
 - Inkrementalna integracja danych między strefami (np. staging → core → marts).
 - Proste wykrywanie insertów, update'ów i delete'ów.

STREAMS

- **Jak to działa?**
 - Tworzysz STREAM na tabeli (lub MV).
 - Snowflake utrzymuje „offset” zmian od momentu utworzenia strumienia albo od ostatniego odczytu.
 - Gdy SELECT-ujesz ze strumienia, dostajesz tylko różnice względem poprzedniego odczytu, z metadanymi o typie zmiany.

STREAMS

- **Kluczowe kolumny metadanych:**
 - METADATA\$ACTION: INSERT albo DELETE
 - METADATA\$ISUPDATE: TRUE dla wierszy będących częścią aktualizacji
 - METADATA\$ROW_ID: wewnętrzny identyfikator wersji wiersza (przydatny do deduplikacji)
- Aktualizacja (UPDATE) pojawia się jako para:
 - DELETE starej wersji + INSERT nowej wersji, obie z ISUPDATE = TRUE.

STREAMS

- **Typy streamów**
 - **Standard stream** (domyślny): śledzi zmiany od momentu utworzenia, “konsumowany” przy każdym pełnym odczycie (po SELECT bez AT/BEFORE jego offset przesuwa się).
 - **Append-only stream:** widzi tylko inserty (brak delete/update). Tańszy i prostszy, gdy zależy Ci tylko na nowych wierszach.
 - **Insert-only na widok materializowany:** dla MV.